Large Language Models as Recommender Systems: A Study of Popularity Bias

Jan Malte Lichtenberg*
jlichten@amazon.de
Amazon Music
Berlin, Germany

Alexander Buchholz*
buchhola@amazon.de
Amazon Web Services
Berlin, Germany

Pola Schwöbel*
schwobel@amazon.de
Amazon Web Services
Berlin, Germany

ABSTRACT

The issue of popularity bias—where popular items are disproportionately recommended, overshadowing less popular but potentially relevant items—remains a significant challenge in recommender systems. Recent advancements have seen the integration of general-purpose Large Language Models (LLMs) into the architecture of such systems. This integration raises concerns that it might exacerbate popularity bias, given that the LLM's training data is likely dominated by popular items. However, it simultaneously presents a novel opportunity to address the bias via prompt tuning. Our study explores this dichotomy, examining whether LLMs contribute to or can alleviate popularity bias in recommender systems. We introduce a principled way to measure popularity bias by discussing existing metrics and proposing a novel metric that fulfills a series of desiderata. Based on our new metric, we compare a simple LLM-based recommender to traditional recommender systems on a movie recommendation task. We find that the LLM recommender exhibits less popularity bias, even without any explicit mitigation.

CCS CONCEPTS

• **Information systems** → **Personalization**; **Language models**; **Information retrieval diversity**; **Recommender systems**.

KEYWORDS

Popularity Bias, Recommender Systems, Large Language Models.

1 INTRODUCTION

Recently, general-purpose large language models (LLMs) have achieved astonishing successes across a wide range of tasks such as summarization, information extraction and content creation. In many domains, LLMs are used as foundation models—multipurpose tools that replace task-specific machine learning models. The broad applicability of LLMs, coupled with their potential for use in intent-following or conversational recommender systems, has spurred interest in exploring their role in this domain [18, 21, 39, 54, 58]. Our work analyzes LLMs as recommender systems along a specific dimension, popularity bias.

Popularity bias occurs when a recommender system disproportionally surfaces popular items (see Sections 2 and 3 for details), and has been present in recommender systems for decades [8, 52]. It is still present today in applications such as modern streaming services that rely on recommender systems to guide the user's consumption and exploration behavior. The potential negative impact of this bias ranges from filter bubbles over reduced user satisfaction, unfairness towards content producers to lost economic opportunity for platform providers. Strategies to quantify, explain, and mitigate popularity bias have seen growing interest lately, see [34] for a recent survey.

Just like with standard recommender systems, popularity-biased behavior would not be unexpected for an LLM-based recommender: during training, the model has likely encountered popular content more often than lesser known content. Training data biases have been shown to propagate into model generations in other contexts such as geographical [48, 61] and gender-occupation biases [28, 37, 43]. On the other hand, LLMs also provide a new opportunity to mitigate popularity bias due to their natural language interface. We can simply *ask* the LLM to recommend more niche content. Such self-debiasing [47] has been proven effective across a range of bias-mitigation tasks [40].

With the aim of quantifying the popularity bias in LLM-based recommender systems, we face a fundamental challenge: the literature lacks a consensus on what is an appropriate metric for measuring popularity bias [34]. Therefore, our first contribution is the development of a principled framework for measuring popularity bias in recommender systems. We start by outlining a set of desiderata that a metric should satisfy, focusing on interpretability and statistical robustness. We then assess existing metrics against these desiderata and introduce a new metric that satisfies them.

To conduct our experiments, we propose a simple LLM-based recommender system, which can be built on top of any general-purpose LLM, and evaluate its popularity bias against a suite of baselines. We then mitigate the bias via prompting and study the resulting accuracy/popularity-bias trade-off.

2 BACKGROUND AND RELATED WORK

2.1 Traditional top-k recommender systems

Traditional top-k recommender systems (RS) serve the purpose of selecting a set of k most relevant items out of a much larger content pool. Common RS such as collaborative-filtering methods [26, 42, 53] operate on *item-level*. That is, these models learn an embedding space representing each candidate item and then recommend the k unseen items that are closest in embedding space to the items previously consumed by the user (item-based collaborative filtering) or items consumed by similar users (user-based collaborative filtering). Different algorithms provide different ways of learning such embedding spaces but they are almost always trained on past user interaction data such as attributed clicks, purchases, or subscriptions.

Several research works have studied popularity bias in traditional recommender systems [1, 13, 34, 59], and have investigated possible sources. Most arguments revolve around the idea that the historical interaction data used to train the recommender system contain,

*The authors contributed equally to this research.

almost by definition, more behavioral signals from popular items than from unpopular ones. Collaborative filtering algorithms then naturally give more weight (on average) to the "naturally more popular" items, unless explicitly avoided by the algorithm [1, 31, 46]. Once such a recommender system is deployed, the popular items receive even more exposure and thus engagement. Consequently, the behavioral data that is gathered to train future generations of the recommender system will be even more popularity biased, resulting in a vicious circle of ever-increasing popularity bias [13].

2.2 Large language models as top-k recommender systems

LLM-based top-k recommenders take a fundamentally different approach in that they are language-based and operate on a *token-level*. In their simplest form, these models generate free-form text recommendations based on a prompt that describes the recommendation task and the user. The recommendations in text form are then parsed and resolved to the respective items in a catalogue before they are shown to the user.

While such models are currently difficult to scale and use in practice due to their slow auto-regressive generation procedure and arguably have not yet reached state-of-the-art performance in traditional recommender evaluation protocols, they provide various advantages that open up entirely new possibilities for recommender systems: First, LLM-based recommenders can be built on top of (open-source) general-purpose foundational models [18, 58], reducing the need for behavioral data collection and model training. [1] Second, LLM-based recommenders can generalize across different content types of a single domain (e.g., music, video, podcast in a media streaming service) or even across domains (e.g., the *P5* model [22]). Cross-domain generalization is notoriously difficult with feature-based recommender systems because of disjoint feature sets and varying consumption across domains. Third, due to their natural-language interface, LLM-based recommenders enable novel paradigms including intent-based recommmenders [9, 41] that allow the user to formulate their complex intent in natural language (e.g., *"I want to watch a New York gangster movie featuring a female lead role"*, *"Movies that likely inspired the cinematographic style of Wes Anderson"*). In conversational recommenders [30], the initial intent-based recommendations can additionally be refined by the user through follow-up requests.

However, as for standard RS, LLMs will be tilted towards what they have learned in their training set, and are prone to predicting popular items at overly high rates, resulting in popularity bias.

2.3 Popularity bias of recommender systems

The literature has produced an abundance of metrics formalizing different interpretations of popularity bias [1, 13, 34, 59, 60]. This diversity in metric selection, while reflective of varied study objectives and applications, poses a challenge in achieving generalizability across research findings, in particular because the normative commitments as to why a particular metric has been selected often remain unstated.

[1]This is particularly true for the movie domain because the training data of most current LLMs is based on the internet, which includes large amounts of movie reviews, discussions, and data bases.

Following Klimashevskaia et al.[34], we start with the general definition of *popularity bias* as a property of a recommender system that is present "*when the recommendations provided by the system focus on popular items to the extent that they limit the value of the system or create harm for some of the involved stakeholders*" [34, p.8].

Stakeholders include the service provider (e.g., a media streaming service or a job board), the content provider (e.g., artists, labels, companies that hire), and the user of the service (e.g. music listeners, job seekers). Popularity bias can have negative effects on all three stakeholders: the user might be bored by repeatedly receiving obvious or unoriginal "mainstream" recommendations; the service provider might suffer customer churn because of their decreased engagement or recommend only items that would have been sold anyways (and thus miss out on long-tail sales [5]); and new or niche content providers might have a hard time entering or be driven out of the market due to limited exposure. Specifically, if a recommender system inordinately promotes items by a group of already popular artists, it may limit exposure of historically disadvantaged groups of content creators, hereby reinforcing this inequality [15]. This is a common concern in the fair ML literature: existing biases are picked up and exacerbated by ML-based systems [7, 51].

Note that while Klimashevskaia et al.'s definition of popularity bias does not prescribe a specific way of how the bias should be quantified, it excludes some existing definitions. For example, unlike in the popularity bias definition by Zhao et al. [60], it is acceptable for some items to be more popular than others a priori. It is equally acceptable to recommend popular items at higher rates; this would constitute popularity bias according to the definition by Abdollahpouri et al. [1] which is reminiscent of the fairness metric demographic parity [12]. Instead, we argue that recommending more popular items more often is acceptable if they indeed are more *relevant* to the user, following an interpretation similar to equality of opportunity [24]. Predicting popular items starts to "*limit the value of the system or create harm for some of the involved stakeholders*" when predicted popularities exceed the user's base popularity in expectation. We will formalize this in the following section.

3 QUANTIFYING POPULARITY BIAS

In working towards reducing the ambiguity of existing popularity bias measures, we propose a framework for defining a popularity bias metric for a given problem and data set. More specifically, we first define a parametrized formulation of popularity bias that generalizes several existing popularity bias metrics for specific parameter settings. We then define a set of theoretical desiderata for an interpretable and statistically robust popularity bias metric and evaluate existing metrics against these standards. Finally, we introduce a new metric that satisfies our desiderata.

As motivated in Section 2.3, we quantify popularity bias of a recommender system with respect to the user's experience. A recommender system is positively (negatively) popularity biased if it recommends popular items at higher (lower) rates than appropriate for that user.

We establish the following basic assumptions on how popularity is measured:

(1) **Data**. The data set provides raw popularity scores $\phi(a)$ for every item a. Raw scores are an aggregate consumption measure over all users (e.g., total number of reviews for a movie, or total number of plays for a song). Online consumption patterns tend to have heavy tails, i.e., a few items account for the major share of interactions. Empirically, item popularity follows a power law (Pareto distribution) arising from "rich get richer" dynamics, [6, 14, 44]. Power laws (and more specifically Pareto-distributions) have probability density functions of the form $p(x) \propto x^{-\alpha}$, for $x \geq x_{min} > 0$, where $\propto$ denotes proportional up to a constant. The coefficient α determines the tail behavior of the distribution. Power laws $\alpha \leq 2$ do not have a finite mean and for $2 < \alpha \leq 3$ the variance is infinite (and consequently all higher moments). Hence, any metric based on averages (like mean, standard deviation, variance, skew and kurtosis) are ill-defined, as empirical averages do not converge. Consequently popularity bias metrics with potentially huge fluctuations will have little scientific value. Therefore, raw popularity bias scores are often transformed implicitly by a function $g(\phi(a)) : \mathbb{R} \to \mathbb{R}$ for all items a. For instance, g normalizes popularity scores to a certain range, or removes heavy tails. These transformations are often done without explanation, a point that we address below.

(2) **Recommender popularity**. The popularity of a top-k recommendation is an aggregate popularity score of all items in the slate,

$$AggP(r) = h\left(\{g(\phi(a))\}_{a \in p_r}\right),$$

where $h : \mathbb{R}^k \mapsto \mathbb{R}$ is a function that maps scores $\{g(\phi(a))\}_{a \in p_r}$ to a single value,[2] $k = |p_r|$, and p_r is the set of all items chosen by the recommender.

(3) **User popularity**. A user's popularity preference is defined by the aggregate popularity score of all (or the last n) items the user consumed in the past (e.g., watch history for a movie streaming service),

$$AggP(u) = h\left(\{g(\phi(a))\}_{a \in p_u}\right),$$

with h as above and p_u denotes the set of items a user interacted with.

To define a complete metric, one therefore has to make choices on (a) how the recommendation popularity $AggP(r)$ and user popularity preference $AggP(u)$ are aggregated on the recommender and user level and how they are combined to calculate the actual bias metric (with a slight abuse of notation, we will denote such a function by $M(u, r)$), and (b) how (if at all) the raw popularity scores are transformed by a function $g(\phi(a)) : \mathbb{R} \to \mathbb{R}$ for all items a. Most popularity bias studies only focus on the definition of M and h, while treating g independently. We argue that the properties of a popularity bias evaluation framework depend jointly on M, h and g.

To make principled choices of M, h, and g, we define a set of desiderata for our final metric. We then discuss existing metrics and define our own metric.

[2] As an example consider the empirical average, that maps scores to one value.

3.1 Desiderata for a popularity bias metric

We define the following desiderata for a popularity bias metric $M(r, u)$ and raw popularity transformation function g.

(1) **Well behaved**. We want our metric to be *well behaved* in a statistical sense, i.e., more data leads to more stable results. Our aggregation function h and transformation g must be chosen such that $AggP(u)$ (and $AggP(r)$ respectively) become more precise with more data (i.e., when $|p_u|, |p_u| \to \infty$). This is crucial for reproducible and coherent statements about the popularity bias of a system. Without this requirement resulting metrics are meaningless as they measure random fluctuation in the data. This desideratum ensures *construct reliabilty* [29].

(2) **Centered around** 0. If $AggP(r) = AggP(u)$ then $M(r, u) = 0$. If there is no difference in aggregate popularity between recommendations and baseline popularity for a user, the metric should map to 0. This requirement influences M, and how we compare the sets p_u and p_r.

(3) **Anti-symmetry**. $M(r, u) = -M(u, r)$. Popularity bias can be "positive" or "negative". While most of the literature is concerned with "positive" popularity bias, i.e., recommending too popular items, "negative" popularity bias can have detrimental effects on the relevance of recommended items. We treat "positive" and "negative" popularity bias equally. We want to discover and mitigate both directions in our experiments. This will again influence the choice of M.

(4) **Sensitivity to the long tail**. Our metric must be robust to large popularity values while at the same time being sensitive. The same absolute difference for large popularity values matters less than the same difference for small popularity values.
We define two sets of recommendations $\bar{r}$ and $\underline{r}$ where the main difference is that $\bar{r}$ has an overall higher popularity than then items in $\underline{r}$. We keep fixed the set of user items u. We then want that

$$|M(\underline{r} \oplus \epsilon, u) - M(\underline{r}, u)| > |M(\bar{r} \oplus \epsilon, u) - M(\bar{r}, u)|,$$

where $\oplus$ denotes an increase of the popularity of one item by $\epsilon > 0$. A shift in popularity for high popularity items of a recommender is relatively smaller than a shift in popularity of less popular items. This will influence all choices of M, h and g. This is important as a small change in the tail of the less popular items should have a relatively stronger influence than an increase in popularity of the most popular items.

(5) **Componentwise monotonicity**. We want a metric monotonic in the popularity scores. If popularity scores are increased by $\epsilon > 0$ for the recommender this should lead to the metric becoming larger (respectively smaller for an increase on the user side). This requirements reads as

$$M(r \oplus \epsilon, u) - M(r, u) > 0,$$

using the same notation as before. This is important as the overall *level* of popularity matters, which is assured by monotonic behavior.